



University of
Sheffield

Composing producer-consumer systems

Zsigmond Kovacs-Nagy

Supervisor: Charles Grellois

*A report submitted in fulfilment of the requirements
for the degree of*

BEng in Computer Science (with Software Engineering)

in the

School of Computer Science

December 8, 2025

Declaration

All sentences or passages quoted in this report from other people's work have been specifically acknowledged by clear cross-referencing to author, work and page(s). Any illustrations that are not the work of the author of this report have been used with the explicit permission of the originator and are specifically acknowledged. I understand that failure to do this amounts to plagiarism and will be considered grounds for failure in this project and the degree examination as a whole.

Name: Zsigmond Kovacs-Nagy

Signature:

Date: December 8, 2025

Abstract

Sequentially composed producer-consumer systems are present in a wide range of fields, including biology, economics, computing and others. Contrary to their prevalence in the real world, there is a lack of academic studies on these sequentially composed systems and their dynamics. As a result, my project aims to provide an empirical study on how these systems work, the ways their subsystems can be composed and the dynamics observed between the subsystems in real time simulations.

As of this review, I have explored relevant literature and started development on the simulations. To showcase the significance of this project, I aim to utilise a utility maximisation algorithm representative of economic models. This will allow the simulation of economics shocks within a system, from which the dynamics of the subsystems can be observed.

Contents

1	Introduction	1
1.1	Aims and Objectives	1
1.2	Overview of the Report	2
2	Literature Survey	3
2.1	Producer-Consumer Systems	3
2.1.1	Historical Context	3
2.1.2	Model Definition	3
2.1.3	Applications	3
2.2	Sequentially Composed Systems	4
2.2.1	Architecture and Applications	4
2.2.2	Dynamics and Behaviours	4
2.3	Economic Approach and Algorithms	4
2.3.1	Network Utility Maximization Framework	5
2.3.2	Primal-Dual Congestion Control Algorithm	5
2.4	Simulation Methodologies	5
2.4.1	Programming Language and Libraries	5
2.4.2	Concurrency: Multithreading vs Multiprocessing	5
2.5	Summary on Survey	6
3	Requirements and Analysis	7
3.1	Analysis of Aims and Objectives	7
3.1.1	Objective 1: Simulation Framework	7
3.1.2	Objective 2: Utility Maximisation Algorithm	7
3.1.3	Objective 3: Dynamics Analysis	8
3.2	Must, Should, Could, Won't Analysis	8
3.2.1	Priorities	8
3.2.2	Must Tasks	9
3.2.3	Should Tasks	9
3.2.4	Could Tasks	9
3.3	Criteria Evaluation and Testing	10
3.3.1	Simulation	10

3.3.2	Experimentation	10
3.4	Ethical, Professional and Legal Issues	10
4	Progress	11
4.1	Simulation Framework	11
4.1.1	Concurrency and parallelism	11
4.1.2	Queue Capacity Constraints	11
4.1.3	Composed Subsystems	12
4.1.4	Production Demand	12
4.2	Utility Maximisation Algorithm	12
4.3	Dynamics Analysis	12
4.3.1	Initial Graph	12
4.3.2	Queue Capacity Graph	12
4.3.3	Rates Instead of Cumulative Values	13
4.3.4	Visible Dynamics	13
5	Conclusions and project plan	14
5.1	Achievements	14
5.2	Plan of Action	14

List of Figures

5.1	Updated 12-week project plan based on Objectives 2 and 3.	15
-----	---	----

List of Tables

Chapter 1

Introduction

Producer-consumer systems follow a pattern where given a shared queue, the producer adds produced items to it and the consumer removes consumed items from it. An example of such a system is the sun emitting photons into the atmosphere and a tree consuming the photons for its internal systems. The pattern for a sequentially composed producer-consumer systems is based on linking multiple producer-consumer systems together in a way where the consumer of one system becomes the producer of another. This can be demonstrated by extending the previous example so that the tree is the consumer of the photon in one system and also the producer of oxygen in another. Composing subsystems like this we can explore the dynamics between parameters like production rate, demand and more to see how an occurrence in one system can affects the others.

Even though there is extensive research on independent producer-consumer systems, there exists a gap in research for the compositions of these systems. Given most systems interact and depend on other systems, exploring this domain could uncover dynamics not yet observed. To support further research, the simulation framework will be made publicly available.

1.1 Aims and Objectives

The primary aim of this project is to investigate the dynamics produced by simulating sequentially composed producer-consumer systems. Additionally, I introduced a secondary aim to apply the simulation to a real world system. Economics shares a significant overlap with the the base simulation, and thus an economics based algorithm was selected to model demand. Using this demand to influence the production rate, the simulation will become more accurate and gain new price parameters useful for observing and influencing dynamics.

Given these aims, there are three main objectives. The first one is developing a simulation framework with the ability to compose producer-consumer systems. The one second is implementing the selected algorithm to provide market based demand within the subsystems. The final one is assessing the dynamics observed and comparing them the the expectations.

1.2 Overview of the Report

This report contains four main chapters.

- Chapter 2: Literature Survey on background of topic, dynamics, algorithms and implementation sources.
- Chapter 3: Requirements and analysis, outlined any areas of concerns and proposed mitigations for any issues. As well as evaluations and priority section.
- Chapter 4: Progress made on implementation of framework and dynamics. Objective 1 achieved.
- Chapter 5: Conclusion was a successful first semester and project plan demonstrated in a Gantt chart.

Chapter 2

Literature Survey

2.1 Producer-Consumer Systems

This section details some background, a definition and applications of producer-consumer systems.

2.1.1 Historical Context

Jeffay [7] gave a definition of a producer and consumer as concurrent processes that must never incur data loss. It also introduced process graphs as a way to visualise real-time problems, simplifying them down, finding bottlenecks and improving efficiency. This proves that using analysing structure can reveal efficiencies.

Szidarovszky [12] provided a mathematical basis to identify the presence of equilibriums and limit cycles based on continuous system. They proved that information lag decreases the stability region. Since my demand algorithm tends towards an equilibrium, knowing the effect of lag will assist in a successful project.

2.1.2 Model Definition

Arpaci-Dusseau and Arpaci-Dusseau [3] provided a very clear explanation of the architecture of a producer-consumer system. They define three core components, a producer that puts items onto a shared queue, a shared queue that connect the producer and consumer via a shared store and a consumer taking items from a shared queue. This definition clearly states the structure and the interactions within the system. This will ensure a smooth implementation.

2.1.3 Applications

In Jeffay [7] the process graph was applied to a latency issue between the hardware in a head mounted display. Using producer-consumer graph and topological analysis a non obvious optimised solution was derived. In Szidarovszky [12] the study was applied to hypothetical

firms producing products based on price for a homogeneous market. The result of extensive analysis showed that if a firm made decisions based on delayed information, it would lead to instability to the extent of potential uncontrolled oscillations in production rates.

2.2 Sequentially Composed Systems

This section outlines how to sequentially compose a producer-consumer system. As well as applications of composed systems and potential dynamics within composed systems.

2.2.1 Architecture and Applications

A sequentially composed producer-consumer system contains a sequence of related producer-consumer subsystems. Each of these subsystem is linked by an adapted consumer with the ability to produce an item to the subsequent queue in the sequence.

Bruns [6] introduced a new term into literature called *produsage*. As the name suggests, it describes a collective that is both a producer and a consumer. The application of produsage does differ in the sense that it's meant to represent crowdsourcing, where the collective works toward incremental improvement to a medium they use. This social network way of composing producers and consumers shows that there is a demand for these systems and there isn't just one way to define them.

Xu et al. [13] introduced a term called DN-PCs representing a Distribution Network producer-consumer system. Given the context of power grids, each individual DN-PC unit in the network is responsible for spreading the load by absorbing and releasing supply across the network via the use of the User Energy Management System (USEM). The ability to both absorb and release power functionally mimics the adapted consumer of a sequentially composed producer-consumer system. This close relation further highlights the applicability of this project.

2.2.2 Dynamics and Behaviours

This paper by Lee et al. [9] explores the mechanism of Bullwhip Effect to showcase how small fluctuations can propagate significant waves higher up the chain. The Bullwhip Effect is an economics version of the Butterfly Effect. The initial fluctuations can be caused by information lag, or by batched updates. Given one of the project aims is to observe the dynamics within the system, the Bullwhip Effect will be kept in mind.

2.3 Economic Approach and Algorithms

I chose to apply my distributed system simulation to the economics sector as there is significant overlap between how the explored systems and economic markets behave.

2.3.1 Network Utility Maximization Framework

This paper by Kelly [8] was the pioneer of Network Utility Maximization NUM. The goal is to maximise the value based off of the utility, price, production rates and queue capacity. This paper also introduced shadow prices depending on queue capacity. Additionally, proportional fairness became reachable gives each independent agent in a system the ability to achieve a global optimum.

2.3.2 Primal-Dual Congestion Control Algorithm

Low and Lapsley [10] developed on top of Kelly's paper. This iteration included the Primal-Dual Congestion Control Algorithm consisting of two sub-loops that maintain prices and source rates.

2.4 Simulation Methodologies

This section covers the reasoning behind the use of Python and multiprocessing for the simulation development. Then moves onto useful resources available to get started with producer-consumer systems and concurrency.

2.4.1 Programming Language and Libraries

Since this project's main aim is to create an empirical study of a programmed simulation, choosing an appropriate programming language is essential. To ensure an appropriate choice, Nanz and Furia's[11] comparative study on different programming languages was leveraged.

The study found that while code from languages like Perl and Python was roughly twice as concise as languages like C/C++ and Java, C still outperformed the other languages in terms of performance. While performance is key, the study noted that the performance of Python is sufficient for most use cases. Using this study as a guide Python was chosen for its fast development and reasonable performance, making it ideal for a dissertation.

Choosing Python provides the benefit of its vast and well supported libraries. For this project the prominent libraries used will be multiprocessing and matplotlib.

2.4.2 Concurrency: Multithreading vs Multiprocessing

Within Python there are two ways to perform multiple tasks concurrently. These are multithreading [2] and multiprocessing[1]. Multithreading uses a single process, has shared memory between its threads and is bound by the Global Interpreter Lock (GIL)[2]. The GIL is responsible for only permitting one thread to run code at a time. Multiprocessing creates multiple processes with individual memory allocations and is not bound by the GIL. Since Multiprocessing is not bound by the GIL, it can run in true parallelism and the access to multiple processes means that it's the preferred choice for CPU-bound tasks.

Given my simulations require continuous processing, the preferred concurrency option is multiprocessing.

Brownlee's resources on multithreading [5] and multiprocessing [4] have proven to be a great foundation for producer-consumer systems within Python. The resource on multithreading [5] introduces the necessary components for a producer-consumer system to operate, including a shared queue, a producer, a consumer and a sentinel value. Integrating these components results in a simple producer-consumer system. The second resource on multiprocessing by Brownlee [4] focuses on resolving the challenge of communication between independent processes. Together, these resources can be used as a basis for the development.

2.5 Summary on Survey

The aim of this project is to see dynamics within sequentially composed producer consumer systems. Within this chapter all the necessary topics for progression were completed.

Chapter 3

Requirements and Analysis

3.1 Analysis of Aims and Objectives

The two primary aims of this project are analysing the dynamics of sequentially composed producer-consumer systems and incorporating a distributed utility maximisation algorithm for each subsystems production rate and demand. Given these two aims, the following three objectives were created.

3.1.1 Objective 1: Simulation Framework

Developing the simulation framework requires the consideration of parallelism, communication and performance. As a consequence of using Python for the simulation, Python's Global Interpreter Lock(GIL) needs to be considered. GIL is responsible for ensuring that only one thread can run code at a time. This means concurrency is possible using multithreading, but to achieve true parallelism in python, multiprocessing needs to be used. Since the simulation will preform a high volume of operation from multiple independent subsystems, multiprocessing was chosen to be the basis of the simulations. Using multiprocessing means that each process has its own memory resulting in a need for a dedicated communication channel between subsystems. The standard communication method between producer-consumer systems is via the use of queues. Fortunately, the multiprocessing library contains a queues implementation and since the use of a queue is standard for these systems I will be using it. Lastly, with regards to performance, the three options were single threading, multithreading and multiprocessing. Having already chosen multiprocessing for it's ability to run in parallel, I am fine with the slight increase in memory overhead compared to multiprocessing.

3.1.2 Objective 2: Utility Maximisation Algorithm

The Primal-Dual Congestion Control algorithm [10] will be implemented for each subsystem to control the production rates. This is crucial since the fluctuations of the production rates will be the basis of the simulation. Since this is a distributed market-based resource allocation

algorithm, there are a few requirements and limitations that need to be discussed. The three main areas of consideration are the update frequency, decentralisation and step size.

Regarding the update frequency of the production rate, the algorithm expects instantaneous updates. However, the implementation will have to rely on a loop to be able to update the price and in turn the production rate. As a result an appropriate frequency of these updates is needed. If the update frequency is too slow, the algorithm might not settle leading to unnecessary oscillations, whereas if the frequency is too high the algorithm's efficiency will degrade and amplify noise. A way to mitigate these issues is by, dynamically updating the update frequency depending on the change in production rates. So that when there are fluctuations in the production rates the update frequency rises and when the rates stabilise the update frequency is reduced. The initial implementation will use a constant update frequency and if time allows the suggested mitigation will be used to increase accuracy.

The distributed nature of the algorithm requires limitations on the data that processes within the subsystems can access. The requirement for the producer processes is that they can only access their own production rate, importance weighting and external price. While the consumers can only access their own capacity, queue length and rate of production to the queue. This results in maximisation of the utility for the whole system without the need for a centralised controller.

Lastly, the value used for the price step size in the algorithm is crucial. If the steps are too low, changes in demand will not be reflected in a timely manner. Whereas if the steps are too high then the price over reacts with respect to demand causing unnecessary oscillations. To avoid these extremes, a similar solution to the one introduced for the update frequency can be used. This will introduce a dynamic approach where higher fluctuations result in lower step sizes. The result of this mitigation will assist in dampening out oscillations and reaching an equilibrium faster. Similar to update frequency, initially a fixed price step will be used, and if time allows the dynamic solution will be implemented.

3.1.3 Objective 3: Dynamics Analysis

To analyse the observed dynamics, two aspects need to be considered: the use of appropriate graphs to showcase the dynamics properly and an understanding of the expected dynamics. Given the simulation will update in real time, producing graphs on the changes in rates is appropriate. One of the most prevalent dynamics we can expect is the Bullwhip Effect. Given that the Bullwhip Effect is an economics version of the Butterfly Effect, the applicability of observing this dynamics would be great.

3.2 Must, Should, Could, Won't Analysis

3.2.1 Priorities

In line with the initial dissertation proposal, the highest priority for this project is the analysis of the dynamics within the simulation. The next priority is the implementation of a utility

maximisation algorithm to better regulate production rate and demand. Lastly, if time permits, exploring different simulation scenarios would prove valuable.

3.2.2 Must Tasks

- Sequential Composition of producer-consumer systems
 - Producer-consumer system: implement a producer and a consumer function where they produce and consume from a shared queue.
 - Composition of system: implement a way for a consumer of one system to become a producer of another.
 - Realistic dynamics: implement a way where demand is prevalent in the systems and influences production metrics.
- Observations of Dynamics
 - Diagrams of the dynamics: plotting graphs of the metrics within the systems to get an understanding of the ongoing dynamics.
 - Explore observed dynamics: understand the underlying dynamics and see if I can make any correlations.
 - Optimal Diagrams: explore options and pick diagrams to present within this report that best showcase the findings being discussed.

3.2.3 Should Tasks

- Utility Maximisation Algorithm
 - Decentralised rate updates: depending on the price defined by the price update equation and utility price.
 - Decentralised price updates: depending on the max capacity and the queue size for each subsystem the price is updated.
 - Diagrams of new values like price and rate: given these values we can simulate how the price of items fluctuate based on supply and demand.
 - Impact of dynamic rates: see the effect of implementing the rates and price update equations.

3.2.4 Could Tasks

- SURE Scheme Application
 - Future work: my supervisor has identified this project as significance study with more potential work available outside the scope of the dissertation. The following Could subsection contains a few of the elements that we would like to undertake

as a part of a SURE Scheme application. Given the successful approval of the SURE scheme there has been discussions of publishing a paper on this study.

- Clustered systems
 - One-to-many or many-to-one subsystems: still having the sequential element but also instead of having a straight line it could be multiple producers or consumers for one counterpart.
- Introducing shocks into the system
 - Results of shocks: Will the dynamics recover or will it spiral out of control? Is there a way to mitigate the effects?
 - Thresholds of shocks: What is the area of stability? What is the threshold for the system to become unstable?
 - Composition influences: what effect will increasing a systems size have on its dynamics? Can any guarantees be provided?

3.3 Criteria Evaluation and Testing

3.3.1 Simulation

For the final submission, each tested system will have a approximately 3 to 5 subsystems. This will ensure that visible propagation of the dynamics, both up and down the sequence.

3.3.2 Experimentation

As a result of the simulation having an economics basis, if time allows there will be a chance to simulate real world scenarios. An example of this could be a sudden enforcement of tariffs on the utility price of specific item during the simulation.

3.4 Ethical, Professional and Legal Issues

This project does not require data collection or external datasets. As a result no ethics approval is needed. All of the produced and used is open sourced.

Chapter 4

Progress

This chapter describes the current progress towards the first objective. To fulfil this objective a simulation framework capable of composing producer-consumer systems was developed.

4.1 Simulation Framework

The simulation framework was developed using Python and integrated libraries such as matplotlib and multiprocessing. Within this section the development of a framework capable of simulating sequentially composed producer-consumer systems is discussed.

4.1.1 Concurrency and parallelism

A simple multithreaded producer-consumer system was developed using Brownlee's[5] guide as a basis. Running the system, the expected production and consumption behaviours were observed in the printed logs. Utilising the multithreading nature of this system, the ability to have multiple producers and consumers for one shared queue was added.

Even though the system worked as expected, the fact that it was using multithreading, meant that it inherited the Global Interpreter Lock (GIL)[2] limitation. As discussed in the requirements the design needed parallelism, which can only be obtained in Python using multiprocessing. This meant the system had to be adapted to use multiprocessing instead of multithreading. To do this another of Brownlee's[4] guides was used. The changes in architecture were minimal, with the main change being the use of the multiprocessing library's own queue, so that processes could communicate without a shared data store. This refactor allowed producers and consumers to run on their own processes in parallel.

4.1.2 Queue Capacity Constraints

An intuitive next step was to add a maximum shared queue size between the producers and consumers. This change was made because the previous system relied on an infinite queue capacity, which is not realistic for these systems. The effect of this change means that in the event of a full queue the producer blocks production until the queue is no longer saturated.

To implement this change the max queue parameter had to be added in the constructor of the multiprocessing queue[1].

4.1.3 Composed Subsystems

Implementing sequential composition into the framework required a way to link systems in such a way that the consumer of one system becomes the producer of another. To accomplish this link, the consumer function was adapted with the ability to add items to another shared queue after consumption. Given this change, the adapted consumer could now behave like both a consumer for its own system and a producer for another. As a demonstration of this functionality, iron ingot and iron plate were selected as linked items that could be modelled using producer-consumer systems. Composing these two systems resulted in a producer and consumer of iron ingots and a consumer of iron plates. In this scenario the iron ingot consumer acts as an iron plate machine that consumes iron ingots to produce iron plates.

Future work within this subsection entails improving the modularity of the composition code. This will allow modifications to the sequence to be made easily.

4.1.4 Production Demand

A simplistic implementation of demand was implemented. Changing the values of the demand function effected the observed dynamics.

4.2 Utility Maximisation Algorithm

Since this section is not covered by my current objective, progress on this will be made at a later date.

4.3 Dynamics Analysis

Within this section, the way in which the simulation dynamics are obtained, showcased and analysed are discussed.

4.3.1 Initial Graph

Using matplotlib the initial graph only recorded the cumulative items produced.

4.3.2 Queue Capacity Graph

Given the importance of the queue capacity, a separate process was added to the run to simply track the queue size. After implementing, two graphs the cumulative and queue capacity were available.

4.3.3 Rates Instead of Cumulative Values

Updating the framework, the simulation now was time dependent instead if item dependent. This meant that now the rate of production made sense to use and was implemented as a graph.

4.3.4 Visible Dynamics

After implementing demand, both me and my supervisor saw an interesting phenomenon where even though the rate was supposed to go up, it went below the baseline and then up. This is something to be kept investigated[9].

Chapter 5

Conclusions and project plan

5.1 Achievements

My goal in terms of progress for the first semester of was to achieve objective 1. Since objective 1 was completed and interesting dynamics could already be visualised, this semester has been classified as a success.

5.2 Plan of Action

The following Gantt table contains my plans for the next semester. Alongside this dissertation, an application to the SURE scheme will be made in the hopes that the scope of this project can be expanded and explored to the fullest.

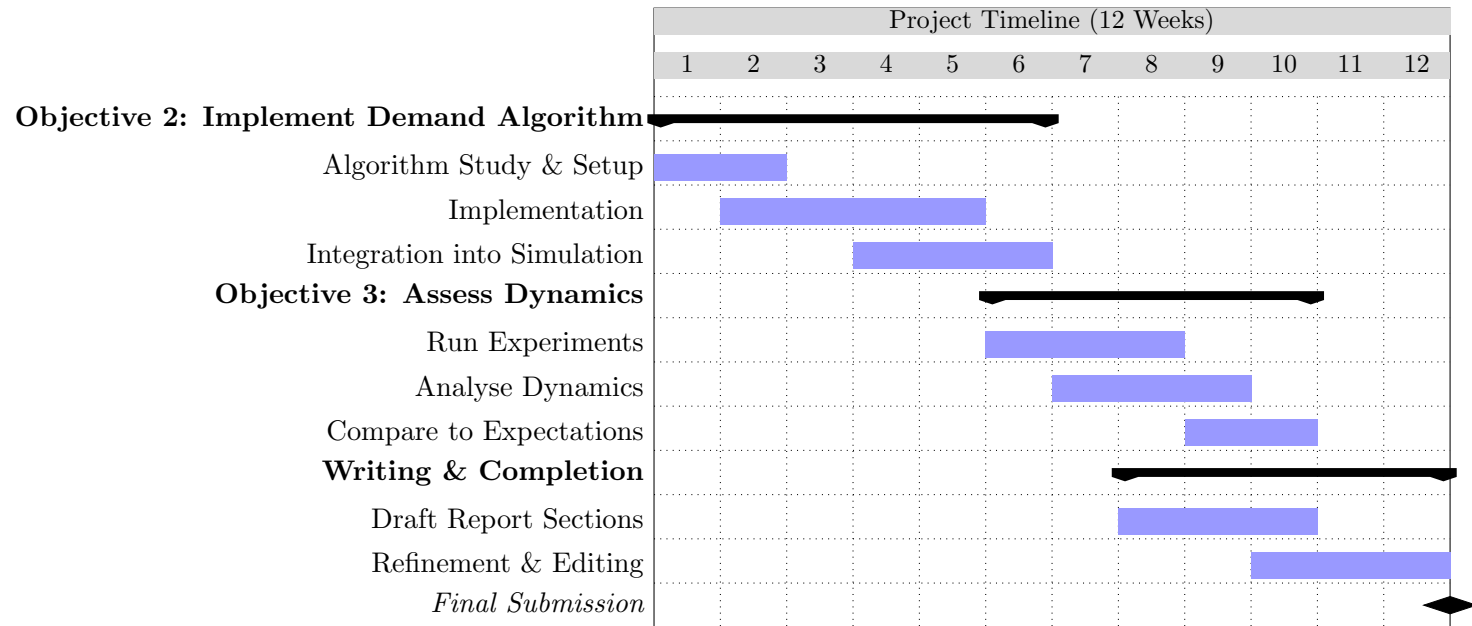


Figure 5.1: Updated 12-week project plan based on Objectives 2 and 3.

Bibliography

- [1] Multiprocessing — Process-based parallelism. <https://docs.python.org/3/library/multiprocessing.html>.
- [2] Threading — Thread-based parallelism. <https://docs.python.org/3/library/threading.html>.
- [3] ARPACI-DUSSEAU, R. H., AND ARPACI-DUSSEAU, A. C. Operating Systems: Three Easy Pieces. In *Operating Systems: Three Easy Pieces*.
- [4] BROWNLIE, J. Multiprocessing Queue in Python, May 2022.
- [5] BROWNLIE, J. Thread Producer-Consumer Pattern in Python, Apr. 2022.
- [6] BRUNS, A. Beyond the Producer/Consumer Divide : Key Principles of Produsage and Opportunities for Innovation. In *The New Development Paradigm: Education, Knowledge Economy and Digital Futures*. Peter Lang, New York, NY, USA, Jan. 2014, pp. 51–65.
- [7] JEFFAY, K. The real-time producer/consumer paradigm: A paradigm for the construction of efficient, predictable real-time systems. In *Proceedings of the 1993 ACM/SIGAPP Symposium on Applied Computing: States of the Art and Practice* (New York, NY, USA, Mar. 1993), SAC '93, Association for Computing Machinery, pp. 796–804.
- [8] KELLY, F. P., MAULLOO, A. K., AND TAN, D. K. H. Rate control for communication networks: Shadow prices, proportional fairness and stability. *Journal of the Operational Research Society* 49, 3 (Mar. 1998), 237–252.
- [9] LEE, H. L., PADMANABHAN, V., AND WHANG, S. Information Distortion in a Supply Chain: The Bullwhip Effect. *Management Science* 43, 4 (Apr. 1997), 546–558.
- [10] LOW, S., AND LAPSLEY, D. Optimization flow control. I. Basic algorithm and convergence. *IEEE/ACM Transactions on Networking* 7, 6 (Dec. 1999), 861–874.
- [11] NANZ, S., AND FURIA, C. A. A Comparative Study of Programming Languages in Rosetta Code. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering* (May 2015), vol. 1, pp. 778–788.

- [12] SZIDAROVSKY, F., AND CHIARELLA, C. The asymptotic behavior of dynamic producer-consumer systems. *Mathematical and Computer Modelling* 39, 11 (June 2004), 1297–1312.
- [13] XU, X., CHENGUANG, H., CHENYA, S., AND JIAN, M. Research on System Resource Optimization of Distribution Network Producer-Consumer System Considering Resilience and Economy. *IEEE Access* 11 (2023), 63947–63957.